

Full-Stack Web Developer Practical Coding Test

Experience: 3+ Years **Suggested Time:** 3–4 Hours

Required Stack: PHP/Laravel + MySQL + React.js/Next.js

Objective

Build a small full-stack Order Management System that demonstrates practical knowledge of Laravel backend development, MySQL, REST APIs, React/Next.js frontend development, authentication, validation, business logic, AJAX/API integration, and clean code.

Technology Requirements

- PHP 8.1+ and Laravel 10/11+
- MySQL
- React.js and/or Next.js
- Tailwind CSS, Bootstrap, or another clean UI approach
- REST API between the frontend and Laravel backend
- Git for version control

Part A — Laravel Backend

1. Authentication & Authorization

- Implement Login and Logout.
- Create Admin and Staff roles.
- Protect private API routes using appropriate Laravel authentication.
- Only Admin can delete products and customers.
- Both Admin and Staff can create and manage orders.

2. Product Management API

Product fields: Product Name, SKU, Category, Price, Stock Quantity, Status.

- Create REST API endpoints for listing, viewing, creating, updating and deleting products.
- Add pagination, search and filtering.
- SKU must be unique.
- Validate all incoming data.
- Do not allow negative price or stock.
- Return appropriate HTTP status codes and JSON responses.

3. Customer Management API

Customer fields: Name, Phone, Email, Address.

- Create REST API endpoints for CRUD operations.
- Implement proper validation.
- Support pagination and search.

4. Order Management & Business Logic

An order must contain a customer and multiple products. Each item should have quantity, unit price and line total.

- Calculate totals on the server.
- Do not trust order totals sent by the frontend.
- Prevent ordering more than available stock.
- Reduce stock after a successful order.
- Restore stock when an order is cancelled.
- Use a database transaction for order + stock changes.
- Use proper relationships between customers, orders, order items and products.

Part B — React.js / Next.js Frontend

5. Login Page

- Create a clean login page connected to the Laravel API.
- Handle loading, validation and API error states.
- Store authentication state securely using an appropriate approach.
- Protect dashboard pages from unauthenticated users.

6. Dashboard

- Show Total Customers, Total Products, Total Orders, Total Sales and Low-stock Products.
- Load dashboard data from the Laravel API.
- Display loading and error states.
- Optional: sales chart by day/month.

7. Product & Customer Pages

- Create list pages with pagination.
- Add search/filter UI.
- Create Add/Edit forms.
- Show validation and API errors clearly.
- Use reusable React components.

8. Order Creation Page

- Select a customer.
- Add multiple products to an order.
- Display current price and available stock.
- Allow quantity changes.
- Calculate the displayed order total.
- Submit the order through the Laravel API.
- Show success/error feedback without unnecessary page reloads.

9. Next.js Requirement

If using Next.js, demonstrate appropriate use of Next.js features such as routing, layouts, server/client components where suitable, or another meaningful Next.js feature.

Part C — Code Quality

- Use clean and maintainable folder structure.
- Use reusable React components.
- Avoid unnecessary duplicate API calls.
- Use meaningful names.
- Keep business logic out of controllers where a cleaner service/domain approach is appropriate.
- Use environment variables for configuration.
- Provide a README with setup instructions.
- Provide database migrations and seeders.
- Include sample login credentials.

Bonus Features

- Laravel Feature/Unit tests for order and stock logic.
- React/Next.js component or integration tests.
- Order invoice/print page.
- REST API documentation.
- Debounced search.
- Role-based frontend navigation.
- Soft deletes.
- Docker setup.
- A dedicated OrderService for order processing.
- Optimistic UI or other thoughtful frontend UX improvements.

Final Technical Question

Scenario: Two customers place orders for the last 5 units of the same product at almost exactly the same time.

Question: What could go wrong with a simple stock check? Explain how you would prevent overselling using Laravel/MySQL concurrency controls and database transactions. Also explain how the React/Next.js frontend should handle an API response indicating that stock is no longer available.

Submission Requirements

- Laravel backend source code.
- React/Next.js frontend source code.
- Database migrations and seeders.
- README with installation/setup instructions.
- API endpoint list.
- Test credentials.
- Git repository with meaningful commits.

Candidate Name: _____

Date: _____ **Time Started:** _____ **Time Finished:** _____